Semester Master Plan and Workflow SOP — calculator update

Prepared October 7, 2026. Local prototype; sample data; no live institutional connection.

Syllabus to calendar

1. Resolve active membership and authorized course context.
2. Reference approved syllabus/course sources; record version, source authority and timezone.
3. Extract candidate milestones, deadlines and grading rules as drafts. Confirm contradictions with the source owner.
4. Maintain one assignment reference shared by master view, Action Center and calendar.
5. Student selects a study block before the confirmed deadline; preview overlap and source freshness.
6. Confirm and save the private block. Scheduling does not submit an assignment.
7. Review the following two weeks at a student-chosen weekly planning time. Reschedule only with student control.
8. Reconcile source changes; preserve private plans and flag conflicts rather than silently rearranging them.

Grade review schedule

At weekly review, refresh released results and read feedback. Load only the selected course into the points calculator. Missing scores are unknown, not zero. Points subtotal = sum known earned / sum corresponding possible × 100.

For weighted categories, use one category subtotal per row, and verify all syllabus weights total 100%. Current estimate = sum(category percentage × category weight) / sum(scored category weight). Show the scored-weight coverage. Do not present a partial normalized result as the final grade.

The prototype does not apply dropped-score, attempt selection, late penalty, rounding, letter-grade or extra-credit rules. Above-maximum earned values are allowed with a warning, not institutional validation.

Course selection clears calculator inputs to prevent inadvertent cross-course mixing. Hypothetical entries remain private and never change released records.

Priority rule

Prototype clock: October 7, 2026 at 12:00 browser-local time. Submitted/graded -> Low. Stale source or active deadline within 72h -> High. Active deadline within 7d -> Medium. Otherwise Low. This is a simple transparent prototype rule, not a verified implementation of Semester's richer next-action ranking. Production uses current time, institutional timezone, documented priorities and student preferences.

Action Center propagation

Command -> trusted identity/ownership/access/source validation -> consequence preview -> confirmation -> transaction saving block, audit and outbox -> authorized consumers -> receipts and reconciliation.

Master view derives planned hours. Calendar creates private time projection. Action Center updates next-action context; assignment submission state stays unchanged. Company sees only a separate sanitized health signal if necessary, not raw personal events.

The HTML simulates this in memory. Production requires transactional writes, idempotent commands, authenticated producers, scope checks, durable retries, atomic consumer deduplication/projection updates, aggregate version checks and recovery owners.

Calendar failure does not undo the saved block. Show Pending, retry with the same event ID, skip consumers already applied. Unknown official/external write outcomes require authoritative reconciliation before retry.

Boundaries

Personal: notes, effort estimates, blocks and what-if grade calculations. Institution: authorized course, submission and released-result records. Company: commercial configuration, minimized support and operational health. One account does not grant universal access. Enforce boundaries at database, storage, search, cache, exports, events and AI retrieval, not solely UI filters.

Verification and activation

File generation, JavaScript syntax and isolated calculator tests checked where runtime available. Browser UX, accessibility, integration, tenant isolation and durable-delivery tests are not run. No deployment or official-record activation claimed.

Engineering owner: reconcile prototype with existing canonical store/services. Academic owner: validate syllabus, deadline and grading rules. Security/privacy owner: approve scopes, classification and any external processing. Operations owner: verify delivery/recovery runbooks.

Production gate: authorization, RLS, grading edge cases, source revisions, timezones/DST, duplicates, concurrency, out-of-order events, recovery and accessibility tests pass; institution authorizes required connections.

Manual acceptance

Sort each heading both ways; filter High and a specific course; inspect no-match state. Plan a non-overlapping pre-deadline block. Simulate calendar failure, disable it and retry; verify no duplicate block and unchanged submission/grades. Reject stale sources, overlaps and after-deadline blocks. Calculate zero earned, blank unknown, zero possible and invalid category weights. Reload and confirm in-memory changes are intentionally discarded.

Evidence: project Grades.jsx separates released grades from estimated course total; Ecosystem.jsx specifies record/audit/outbox and authorized consumers; SEMESTER_NEXT_ACTION_RANKING.md describes a richer design-stage ranking model. These are design evidence, not production verification.